

Full-Stack Assignment: Vehicle Service Management System

Objective:

The goal of this assignment is to develop a full-stack web application for a **Vehicle Service Management System**. The application should provide functionalities for managing vehicle services, repairs, components, pricing, and revenue calculation.

Requirements:

1. Component Registration & Pricing Management

- As an operations user, you should be able to register new components, including their repair pricing or purchase details. This will allow the system to track parts used for repairs or new vehicle services.

2. Vehicle Repair Tracking

- Users should be able to add and manage details for vehicles that require repairs. This includes entering information such as vehicle identification, issue description, and repair requirements.

3. Issue Reporting and Component Selection

- Users will be able to add issues related to a vehicle. They should have the option to choose between using new components or opting for repair services to resolve the issue.

4. Final Price Calculation & Payment

- After selecting components and services, the system should calculate the final price for the repair or service based on selected parts, labor, and any other applicable charges.
- While real-world payment integrations are not required, you should simulate the payment process by calculating the amount due for services.

5. Revenue Graphs

- Implement graphs to visualize daily, monthly, and yearly revenue from repairs and services. These graphs should be responsive and adapt to various screen sizes.
 - Recommended library: [Recharts](#), though other libraries like Chart.js can be used as well.
-

Bonus Points:

- **Unit Tests:** Bonus points will be awarded for writing unit tests that cover important aspects of the application, such as models, views, and forms.
-

Tech Stack:

- **Backend:** Django (for API and server-side logic)
 - **Frontend:** React.js (for dynamic, responsive user interfaces)
 - **Database:** SQLite or PostgreSQL (Django's default ORM can be used for database management)
 - **Charts Library:** Recharts or any other library for visualizing revenue data
-

Submission Guidelines:

1. **GitHub Repository:**
 - Create a GitHub repository containing the complete codebase.
 - Make sure your code is well-organized and follows best practices.
 2. **README File:**
 - Provide clear instructions on how to set up and run the project locally. This should include any installation dependencies, setup steps, and how to run the backend and frontend servers.
 3. **Demonstration:**
 - Include screenshots or a short video demonstrating the working application, showcasing features such as:
 - Registering components and vehicles.
 - Adding vehicle issues and choosing components.
 - Calculating the final price and simulating a payment.
 - Displaying the revenue graphs.
-

Timeframe:

- You have **24 hours** to complete and submit the assignment.
-

Evaluation Criteria:

- **Functionality:** The system should meet all the requirements outlined above.
 - **Code Quality:** The code should be clean, well-structured, and easy to maintain.
 - **User Interface:** The frontend should be responsive, intuitive, and user-friendly.
 - **Testing:** Bonus points for writing unit tests that verify the functionality of critical parts of the application.
-

Important Notes:

- Ensure that your application is fully functional and all required features are implemented.
- Focus on both the backend logic (Django/FastApi) and the frontend user experience (React).
- Proper error handling, validation, and a good user experience are essential for a complete solution.

Good luck, and feel free to reach out if you have any questions during the development process!